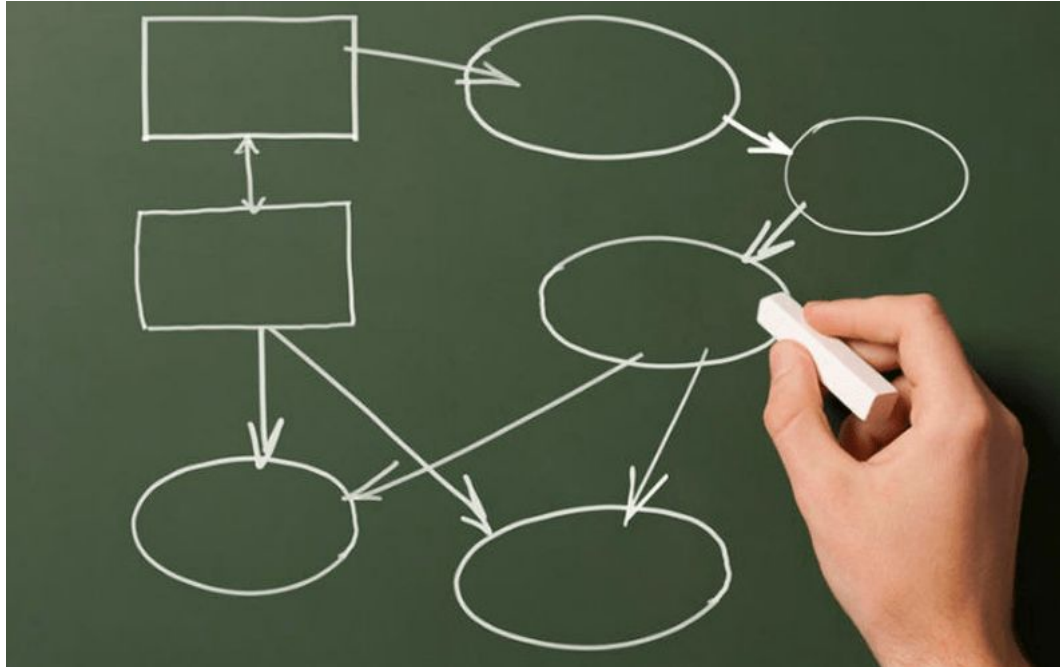
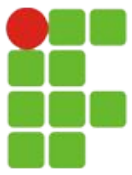
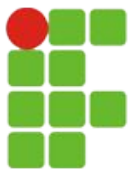




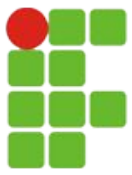
Algoritmos e programação– ALG

Prof. Jones

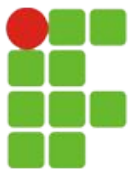


Recordando

- ❖ **Desvio simples (SE)**
- ❖ **Desvio composto (SE / SENÃO)**
- ❖ **Desvios encadeados (SE / SENÃO SE)**
- ❖ **Desvio com condição múltipla (CASO)**



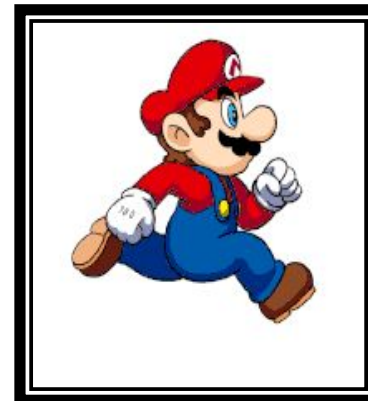
Estruturas de repetição

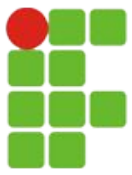


O que é uma repetição na computação?

Estruturas de repetição permitem que um conjunto de instruções sejam executadas repetidas vezes enquanto uma condição for verdadeira.

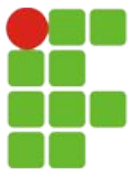
Na computação ela pode ser comumente chamada de **laços** ou **loopings**.





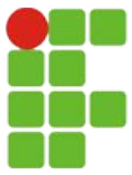
Estrutura de repetição

- Para se criar um **laço de repetição** devem ser definidos:
 - Uma forma de indicar o retorno a um ponto determinado do código;
 - Um recurso para registrar o número de vezes que o laço foi executado, ou seja, a utilização de um **contador** que deverá ser alterado cada vez que o laço for executado;
 - uma **condição** que será testada ao final/início da execução do laço permitindo determinar se ele será repetido ou não.

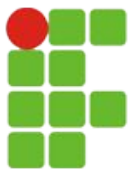


Tipos de estrutura de repetição

- ✓ Repetição Condicional **Faça Até** (pós-teste)
- ✓ Repetição Condicional **Enquanto Faça** (pré-teste)
- ✓ Repetição Contada **Para Até Faça** (iterativo)
 - ✓ Na linguagem C++ temos mais 2 estruturas de repetição que são utilizadas para containers:
 - ✓ Para cada (for_each)
 - ✓ Para automático (for auto)



Pré-teste



Estrutura de repetição (**Pré-teste**)

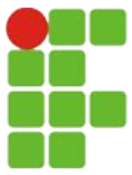
O comando utilizado para repetição com avaliação prévia (**teste de condição no início**) é: ***while* ()**

- Condição é testada **antes** de se iniciar o bloco de instruções;
- Se resultado da condição for (**F**) o bloco de instruções não será executado **nenhuma vez**.

Teste no início



```
while (<condição>)  
{  
    [bloco de instruções]  
}
```



Problemas reais de computação

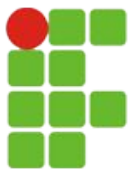
Como seria um algoritmo para exibir na tela todos números entre **1 e 10**?



```
#include<iostream>

using namespace std;

int main()
{
    cout << "1" << endl;
    cout << "2" << endl;
    cout << "3" << endl;
    cout << "4" << endl;
    cout << "5" << endl;
    cout << "6" << endl;
    cout << "7" << endl;
    cout << "8" << endl;
    cout << "9" << endl;
    cout << "10" << endl;
    return 0;
}
```



Problemas reais de computação

Como seria um algoritmo para exibir na tela todos números entre **1 e 10**?

```
#include<iostream>

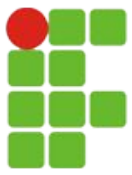
using namespace std;

int main()
{
    int N = 1;
    while (N<=10)
    {
        cout << N << endl;
        N = N + 1; // ou N++
    }
    return 0;
}
```

Problemas reais de computação

Mostrar o seu nome completo 10 vezes na tela?

[illegible]



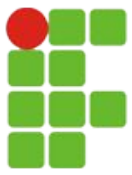
Problemas reais de computação

Mostrar o seu nome completo **10** vezes na tela?

```
#include<iostream>

using namespace std;

int main()
{
    int N = 1;
    while(N < 10)
    {
        cout << "Jones Mendonça de Souza" << endl;
        N = N + 1;
    }
    return 0;
}
```

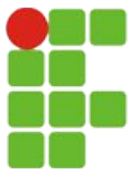


1.ii. while()

- Problema - Senha fixa:

Escreva um programa que repita a leitura de uma senha até que ela seja válida. Para cada leitura de senha incorreta informada, escrever a mensagem "Senha Invalida". Quando a senha for informada corretamente deve ser impressa a mensagem "Acesso Permitido" e o algoritmo encerrado. Considere que a senha correta é o valor 2002.

Exemplo de Entrada	Exemplo de Saída
2200	Senha Invalida
1020	Senha Invalida
2022	Senha Invalida
2002	Acesso Permitido



- Problema - Senha fixa:

1.ii. while()

Estudo do enunciado:

- Entradas: senha
- Saídas: mensagem “**Senha Invalida**” OU mensagem “**Acesso Permitido**”
- Operação: verificar se senha é igual ou não a 2002

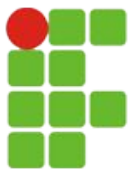
Estrutura de seleção: if ()

Estrutura de repetição: while ()

```
#include<iostream>

using namespace std;

int main()
{
    int senha = 0;
    while(senha != 2002)
    {
        cin >> senha;
        if(senha == 2002)
            cout << "Acesso Permitido" << endl;
        else
            cout << "Senha Invalida" << endl;
    }
    return 0;
}
```



- Problema – Senha fixa:

1.ii. while()

```
#include<iostream>

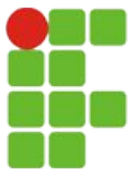
using namespace std;

int main()
{
    int senha = 0;
    while(senha != 2002)
    {
        cin >> senha;
        if(senha == 2002)
            cout << "Acesso Permitido" << endl;
        else
            cout << "Senha Invalida" << endl;
    }
    return 0;
}
```

while()

condição para repetição

instruções que serão
repetidas



▪ Problema - Senha fixa – Teste de Mesa:

1.ii. **while()**

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int senha = 0;
```

```
    while(senha != 2002)
```

```
    {
```

```
        cin >> senha;
```

```
        if(senha == 2002)
```

```
            cout << "Acesso Permitido" << endl;
```

```
        else
```

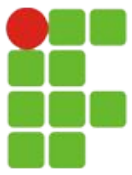
```
            cout << "Senha Invalida" << endl;
```

```
    }
```

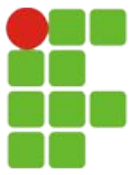
```
    return 0;
```

```
}
```

Passo	Linha	senha	Condição	Saída
1	5	0	-	-
2	6		(0 != 2002) ? V	-
3	8	[2200]	-	-
4	9		(2200 == 2002) ? F	-
5	12		-	Senha Invalida
6	6		(2200 != 2002) ? V	-
7	8	[2002]	-	-
8	9		(2002 == 2002) ? V	-
9	10		-	Acesso Permitido
10	6		(2002 != 2002) ? F	-
11	14			



Pós-teste



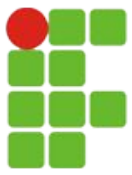
Estrutura de repetição (**Pós-teste**)

O comando utilizado para repetição com avaliação posterior (**teste de condição no fim**) é: ***do while()***

- Condição é testada **após** a execução do bloco de instrução;
- Bloco de instruções **será executado pelo menos uma vez.**

```
do {  
  
    [bloco de instruções]  
  
} while (<condição>);
```

Teste no fim



Problemas reais de computação

Como seria um algoritmo para exibir na tela todos números entre **1 e 10 usando (do while)**?

```
#include<iostream>

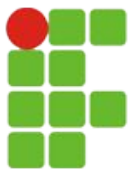
using namespace std;

int main()
{
    int N = 1;
    do
    {
        cout << N << endl;
        N = N + 1;
    } while (N <= 10);
    return 0;
}
```

```
#include<iostream>

using namespace std;

int main()
{
    int N = 1;
    while (N<=10)
    {
        cout << N << endl;
        N = N + 1; // ou N++
    }
    return 0;
}
```



Problemas reais de computação

Mostrar o seu nome completo **10** vezes na tela?

```
#include<iostream>

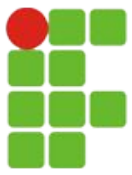
using namespace std;

int main()
{
    int N = 1;
    do
    {
        cout << "Jones Mendonça de Souza" << endl;
        N = N + 1;
    }while(N <= 10);
    return 0;
}
```

```
#include<iostream>

using namespace std;

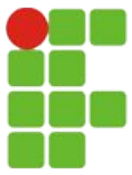
int main()
{
    int N = 1;
    while(N < 10)
    {
        cout << "Jones Mendonça de Souza" << endl;
        N = N + 1;
    }
    return 0;
}
```



Cuidado!!!

As estruturas **while()** e **do while()** são controladas por meio de uma expressão de avaliação. Durante a execução das estruturas (em cada passada), o valor da variável de controle deve sofrer mudança ao longo da execução, para que algum momento pare de repetir.

Caso você não tenha este cuidado, entrará em um problema de **LOOP INFINITO** ou **Stack Overflow** ou nas competições de programação competitiva **Time Limit Exceeded/ Run Time Error**.



Qual a diferença entre **while()** e **do while()** ?





Repetição por **Salto**



Repetição por Salto

goto

- ❑ O **goto** é o último comando de controle de fluxo.
- ❑ Ele pertence a uma classe à parte: a dos comandos de salto incondicional.
- ❑ O **goto** realiza um **salto** para um local especificado. Este local é determinado por um **rótulo**.
- ❑ Um **rótulo**, na linguagem C, é uma marca no programa. Você dá o nome que quiser a esta marca.

```
nome_rótulo  
  
    instruções  
    ...  
goto nome_rótulo;
```



Repetição por Salto

Exemplo:

```
#include<stdio.h>

int main()
{
    int op;
    volta:
        printf("MENU DO SISTEMA\n");
        printf("OPÇÃO 1 - Cadastrar cliente\n");
        printf("OPÇÃO 2 - Pesquisar cliente\n");
        printf("OPÇÃO 3 - Sair do sistema\n");
        printf("Digite a opção desejada: ");
        scanf("%d", &op);
        if(op >=1 && op <=3) goto volta;

    return 0;
}
```



Repetição por Salto

Exemplo:

```
#include<stdio.h>

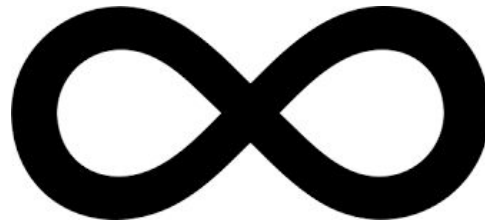
int main()
{
    int op;
    volta:
        printf("MENU DO SISTEMA\n");
        printf("OPÇÃO 1 - Cadastrar cliente\n");
        printf("OPÇÃO 2 - Pesquisar cliente\n");
        printf("OPÇÃO 3 - Sair do sistema\n");
    invalida:
        printf("Digite a opção desejada: ");
        scanf("%d", &op);
        if(op >=1 && op <=3) goto volta;
        if(op <1 || op > 3) goto invalida;

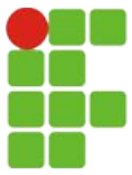
    return 0;
}
```



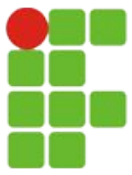
Medidas de precaução com as estruturas de repetição

- Prestar atenção na condição de parada;
- Prestar atenção nas instruções que for colocada dentro do bloco de repetição;
- Um loop infinito pode ocasionar em estouro de memória □ comumente utilizado para criação de softwares maliciosos (vírus)





EOF (End Of File)
Fim de arquivo



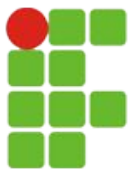
EOF

Alguns problemas pedem que a entrada termine em EOF. Significa que, o seu programa deve realizar a leitura das entradas até encontrar o **FINAL DO ARQUIVO**. Para fazer isso, precisamos percorrer todas as entradas usando **Loop**, **ATÉ** que a condição seja **FALSA**, ou seja, a condição de parada é quando o compilador retornar 0 (FALSO). Neste caso, basta colocar o fluxo de entrada **cin** na condição do Loop.

```
#include <iostream>

using namespace std;

int main()
{
    int entrada;
    while(cin >> entrada)
    {
        //lógica do exercícios com as entradas
    }
    return 0;
}
```



EOF

```
#include <iostream>

using namespace std;

int main()
{
    string a;
    while (cin >> a)
    {
        //Lógica do problema
    }
}
```

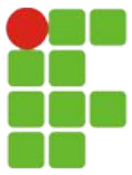
Leitura de strings

```
#include <iostream>

using namespace std;

int main()
{
    string a;
    while (getline(cin, a))
    {
        //Lógica do problema
    }
}
```

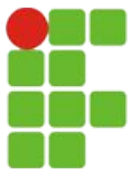
Leitura de strings com espaços em branco



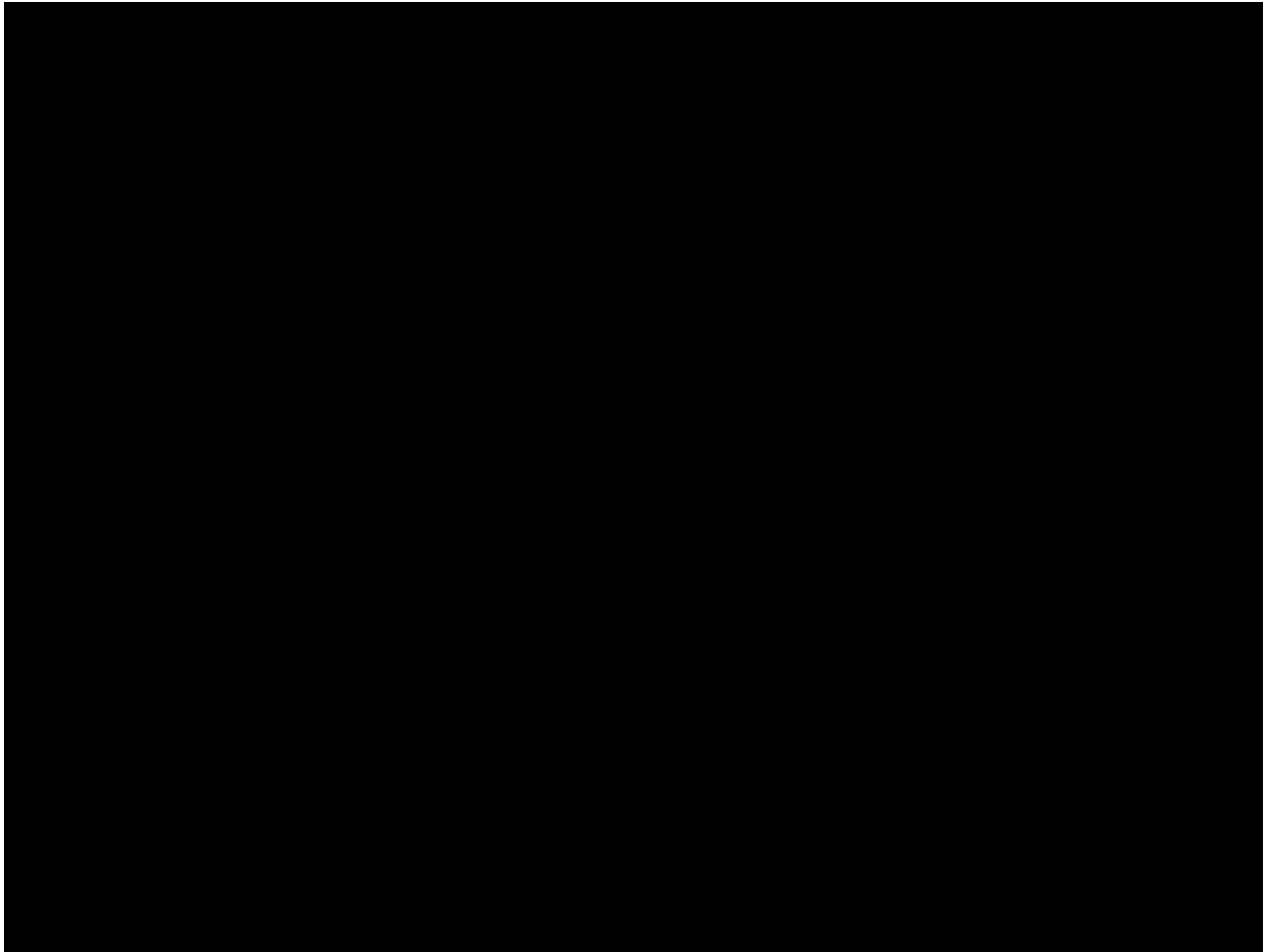
EOF

Algumas vezes, até o algoritmo percorrer o final do arquivo, podemos nos deparar com dados de tipos diferentes na leitura do EOF. Para isso, precisamos limpar o buffer da memória, para não guardar sujeira, usando o `cin.ignore();`
O uso do `cin.ignore()` deve acontecer quando há leitura de valores numéricos seguidos do `getline()`

```
int ID, IDADE;  
string NOME;  
  
cin >> ID;  
cin.ignore();  
getline(cin, NOME);  
cin >> IDADE;
```



Um bom treinamento para REPETIÇÃO (MÚSICAS)



Repetição condicional

5 patinhos foram passear
Além das montanhas
Para brincar
A mamãe gritou: Quá, quá, quá, quá
Mas só 4 patinhos voltaram de lá.

4 patinhos foram passear
Além das montanhas
Para brincar
A mamãe gritou: Quá, quá, quá, quá
Mas só 3 patinhos voltaram de lá.

3 patinhos foram passear
Além das montanhas
Para brincar
A mamãe gritou: Quá, quá, quá, quá
Mas só 2 patinhos voltaram de lá.

Usar um Desvio, para atender o singular

2 patinhos foram passear
Além das montanhas
Para brincar
A mamãe gritou: Quá, quá, quá, quá
Mas só 1 patinho voltou de lá.

Usar um Desvio, para atender o singular

1 patinho foi passear
Além das montanhas
Para brincar
A mamãe gritou: Quá, quá, quá, quá
Mas nenhum patinho voltou de lá.

A mamãe patinha foi procurar
Além das montanhas
Na beira do mar
A mamãe gritou: Quá, quá, quá, quá
E os 5 patinhos voltaram de lá.

Podemos perceber na música, que há um padrão de repetição, então conseguimos resolver o problema utilizando as estruturas que abordamos em sala de aula.

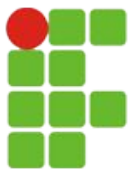
Vamos fazer juntos agora?

Neste caso o cliente pediu um programa para ler a quantidade de patinhos que deseja na música, e o programa deve escrever a letra da música.

```

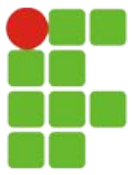
1  #include<iostream>
2  #include<locale>
3
4  using namespace std;
5
6  int main()
7  {
8      setlocale(LC_ALL, "Portuguese");
9      int n_patinhos, controle;
10     cin >> n_patinhos;
11     controle = n_patinhos;
12
13     while(controle >= 1)
14     {
15         if(controle != 2 && controle != 1)
16         {
17             cout << controle << " patinhos foram passear" << endl;
18             cout << "Além das montanhas" << endl;
19             cout << "Para brincar" << endl;
20             cout << "A mamãe gritou: Quá, quá, quá, quá" << endl;
21             cout << "Mas só " << controle-1 << " patinhos voltaram de lá." << endl << endl;
22         }
23
24         if(controle==2)
25         {
26             cout << controle << " patinhos foram passear" << endl;
27             cout << "Além das montanhas" << endl;
28             cout << "Para brincar" << endl;
29             cout << "A mamãe gritou: Quá, quá, quá, quá" << endl;
30             cout << "Mas só " << controle-1 << " patinho voltou de lá." << endl << endl;
31         }
32         if(controle==1)
33         {
34             cout << controle << " patinho foi passear" << endl;
35             cout << "Além das montanhas" << endl;
36             cout << "Para brincar" << endl;
37             cout << "A mamãe gritou: Quá, quá, quá, quá" << endl;
38             cout << "Mas nenhum patinho voltou de lá." << endl << endl;
39         }
40
41         controle--;
42     }
43
44     cout << "A mamãe patinha foi procurar" << endl;
45     cout << "Além das montanhas" << endl;
46     cout << "Na beira do mar" << endl;
47     cout << "A mamãe gritou: Quá, quá, quá, quá" << endl;
48     cout << "E os " << n_patinhos << " patinhos voltaram de lá." << endl;
49     return 0;
50 }

```



2º treinamento para REPETIÇÃO (MÚSICAS)





Mariana (While vs Goto)

Versão usando **While**

```
#include <iostream>

using namespace std;

int main () {

    int conta, i = 1;
    cin >> conta;

    while(i <= conta)
    {
        cout << "Mariana conta " << i << endl;
        cout << "Mariana conta " << i;
        if(i==1)
        {
            cout << ", eh 1, eh 1";
        }
        else
        {
            int repete = i, inicio=1;
            while(inicio <= repete)
            {
                cout << ", eh " << inicio;
                inicio++;
            }
        }
        cout << ", eh" << endl;
        if(i == conta)
            cout << "Ana, viva a Mariana, viva a Mariana" << endl;
        else
            cout << "Ana, viva a Mariana, viva a Mariana" << endl << endl;
        i++;
    }
    return 0;
}
```

Versão usando **Goto**

```
#include <iostream>

using namespace std;

int main () {

    int conta, i = 1, k_2=1;
    cin >> conta;

    refrao:
        int k = 1;
        cout << "Mariana conta " << i << endl;
        cout << "Mariana conta " << i;
        repete:
            cout << ", eh " << k;
            k_2++;
            if(k_2>=1 && k_2<=2) goto repete;
            k++;
            if(k>=1 && k<=i) goto repete;
            cout << ", eh" << endl;
            if(i==conta)
                cout << "Ana, viva a Mariana, viva a Mariana" << endl;
            else
                cout << "Ana, viva a Mariana, viva a Mariana" << endl << endl;
            i++;
            if(i>=1 && i<=conta) goto refrao;
        return 0;
}
```

1) Faça o algoritmo que mostre os múltiplos de 5 de 0 a 50.

5 10 15 20

2) Faça um algoritmo que leia a idade de 10 pessoas e mostre a média das idades destas pessoas.

Dica: A soma das idades sempre será inteiro, mas a média poderá ser um valor não inteiro, portanto necessita fazer a conversão de tipo!!!

3) Faça um algoritmo que mostre a tabuada do 6 utilizando a estrutura de repetição. O programa deverá ser semelhante com a figura a seguir.

```
1.   x 6 = 6
2.   x 6 = 12
3.   x 6 = 18
4.   x 6 = 24
5.   x 6 = 30
6.   x 6 = 36
7.   x 6 = 42
8.   x 6 = 48
9.   x 6 = 54
10.  x 6 = 60
```

4) A disciplina de ALG calcula a sua média bimestral a partir da média aritmética das notas de 1 provas e 1 trabalho. Por exemplo, um aluno que tirou na prova a nota 5,00 e no trabalho a nota 7,00 tem a sua média bimestral de 6,00. Escreva um ALGORITMO que irá calcular a média bimestral para os 20 alunos da disciplina de ALG dando como entrada a nota de 1 prova e 1 trabalho. Caso possível executar um algoritmo a tela de apresentação deveria parecer com a do exemplo abaixo:

```
Entre com a nota da prova: 5,00
Entre com a nota do trabalho: 7,00
Sua média bimestral eh 6,00
....
....
....
Entre com a nota da prova: 8,00
Entre com a nota do trabalho: 9,00
Sua média bimestral eh: 8,50
```